Sales Forecasting
Rossmann Store Sales

Presented By

- Nermin Ahmed
- Donia Ahmed
- Naira Magdy
- Engy Samy
- Reem Radwan
- Khaled Ahmed (Team Leader)

INTODUCTION

Benefits of Forecasting for Any Business Model

- Forecasting is the process of estimating or predicting future events based on historical data and current analysis. It is a critical element that ensures the sustainability and growth of any project.

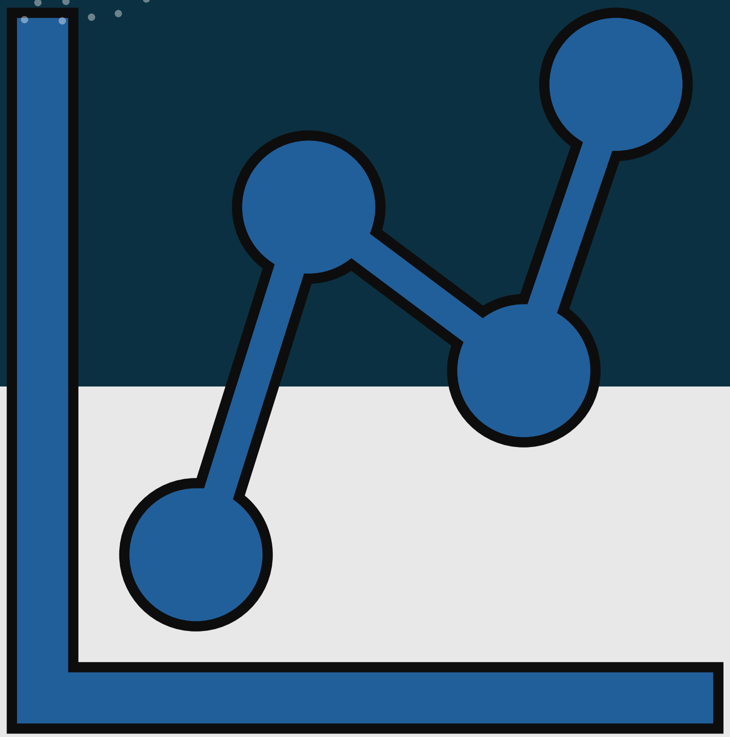
Importance of Sales Forecasting for Businesses

- Improved Inventory Management
- Better Financial Planning
- Efficient Workforce Planning
- Smarter Promotion & Marketing Decisions
- Risk Reduction
- Data-Driven Decision Making

Sales Forecasting Overview

- Sales forecasting for Rossmann stores involves using historical sales data and various factors like promotions, holidays and competitor activity to predict future daily sales, often for up to six weeks in advance.

Scope & Goals



- The dataset contains historical sales data for 1,115 Rossmann stores. The primary objective is to forecast the "Sales" column for the test set.

Data Cleaning

Dataset Overview

- Dataset:
  -Rossmann Sales Forecasting
- Training Data Shape:
  -1,017,209 rows × 9 columns
- Key Features:
  -Numerical, Categorical, Date
- Store Data Shape:
  =1119 rows × 10 columns

Data Quality Problems

Large real-world dataset

- requires preprocessing

Issues found after merging with store data:

- Missing values in : (Competition Distance, Competition Open Since Month/Year, Promo2 Since Week/Year, Promo Interval)

Categorical columns not numerical

Different feature scales

Outliers in Sales

Missing Values & Encoding

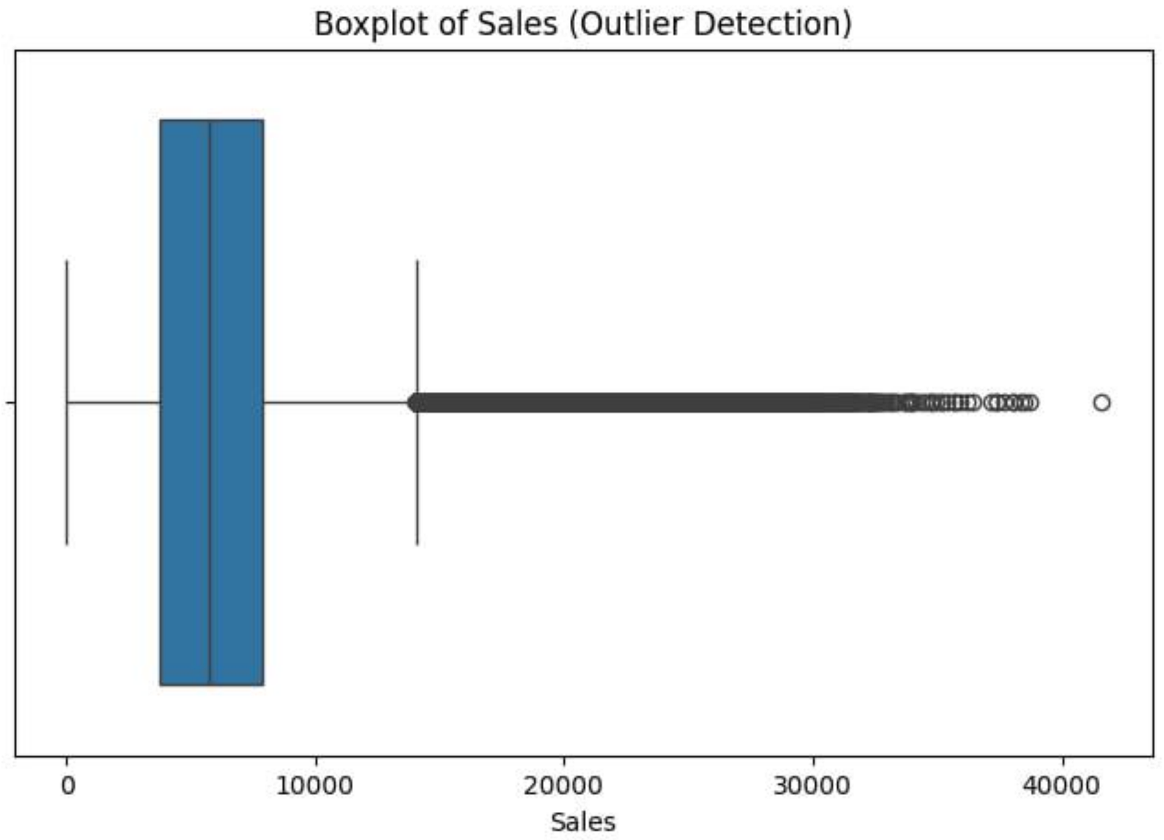
- **Missing Values Handling:**
- Categorical (Promo Interval): filled with Most Frequent Value
- Numerical: filled using Iterative Imputer (MICE)

- **Encoding Applied:**

  - Label Encoding:
    State Holiday (contained “0”, a, b, c)→ numeric values
  - One-Hot Encoding :
    Promo Interval → Monthly flags
    (Promo_Jan, Promo_Feb,… Promo_Dec)
    Binary columns created for each month .

Scaling & Outlier Handling

- **Min-Max Scaling:**
  - Normalizes numerical features to same range
  - Prevents dominance of large-scale features (Sales, Customers)
- **Outlier Detection:**
  - Feature: Sales
  - Method: IQR (Interquartile Range)
  - Outliers: 26,694 (2.62%)

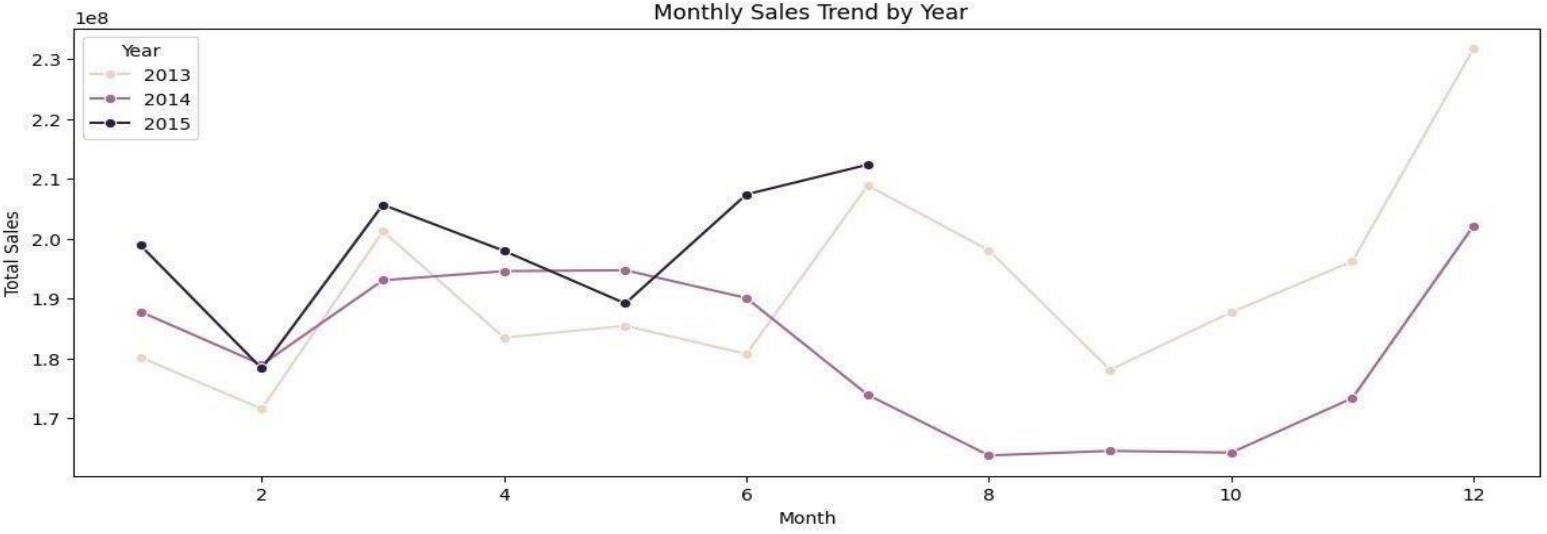


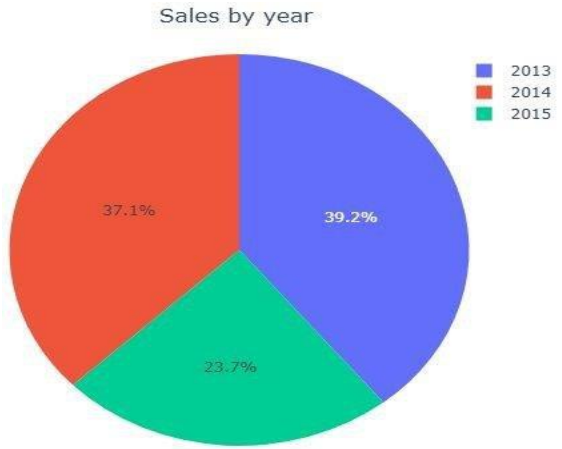
Data Visualization

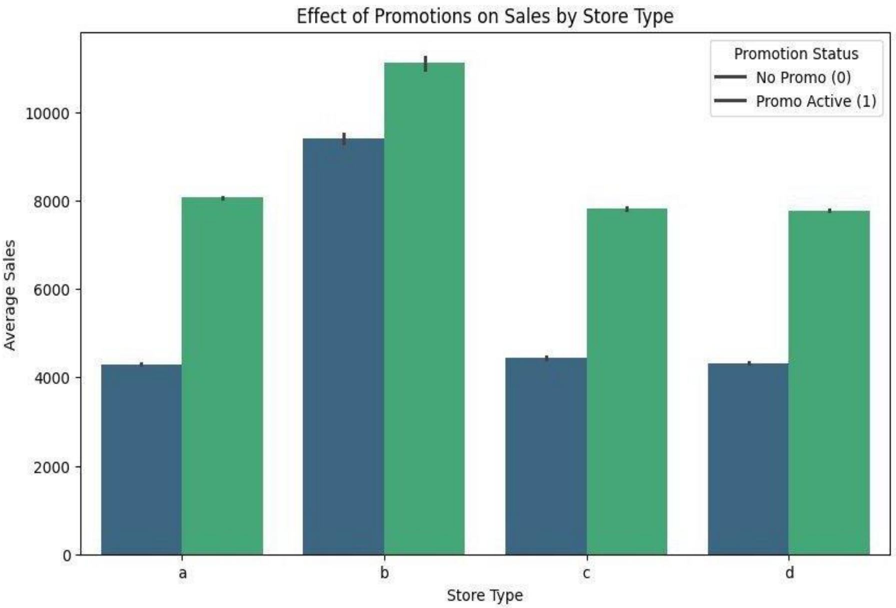
Total Sales
5,873,180,623

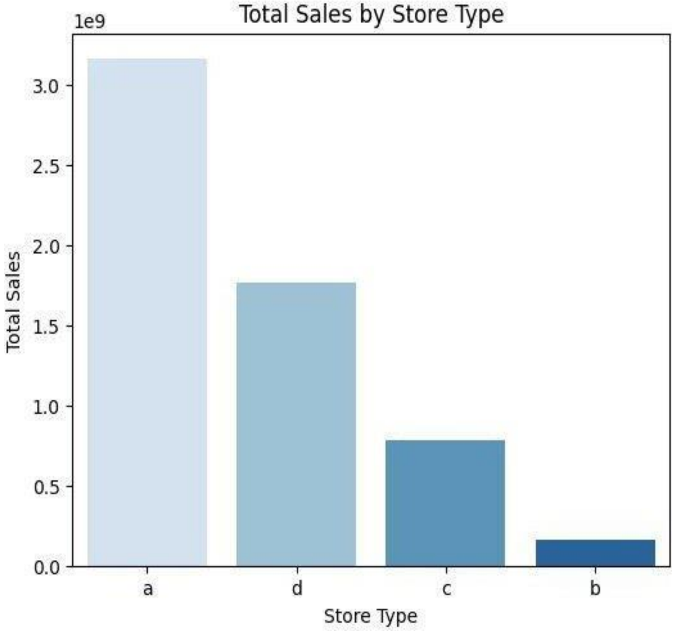
Avg Daily Sales
5,774

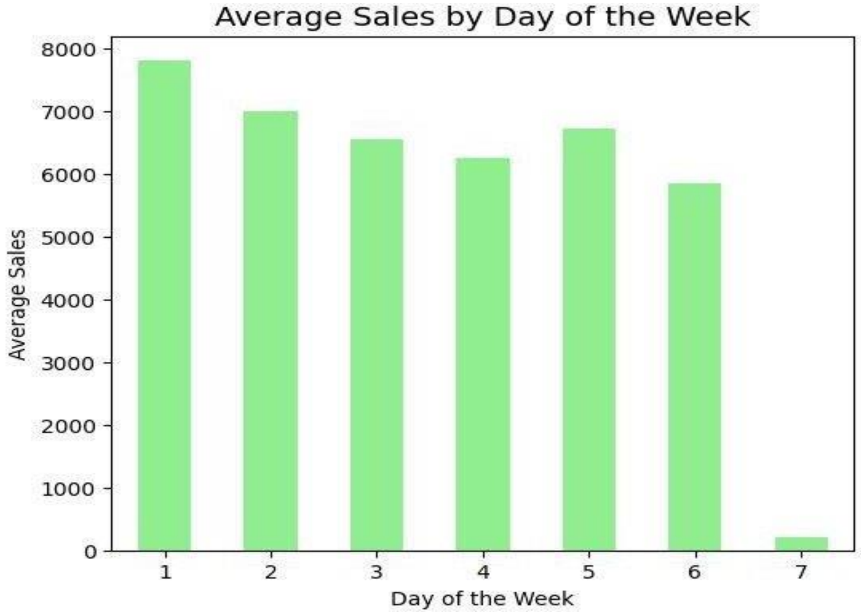
Active Stores
1115



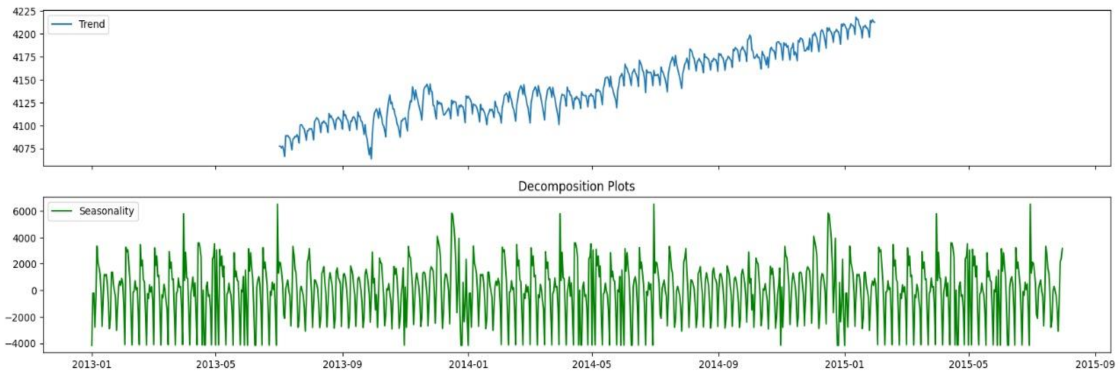




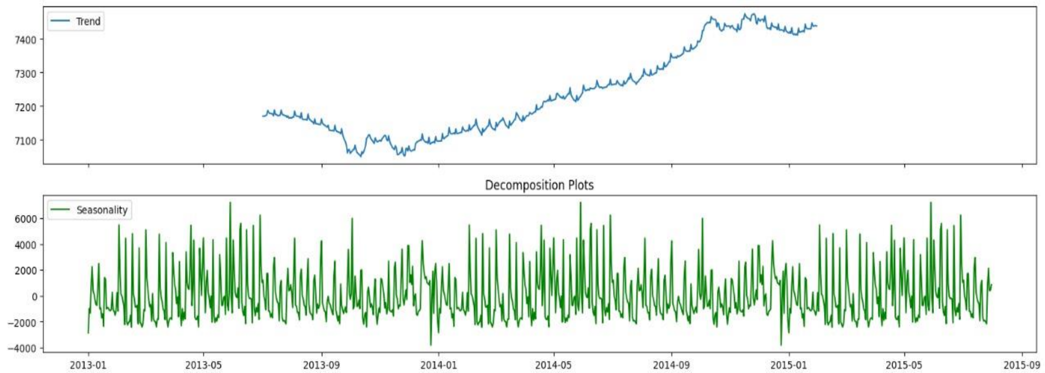




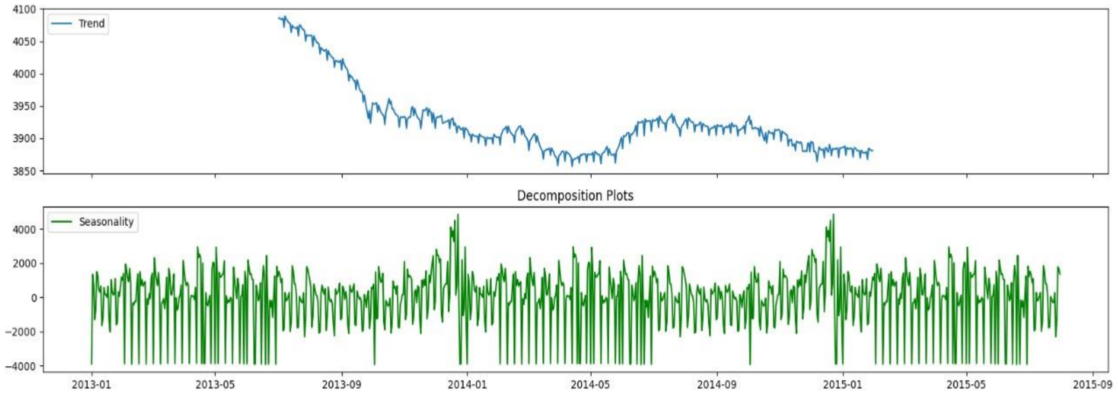
Trend and Seasonality



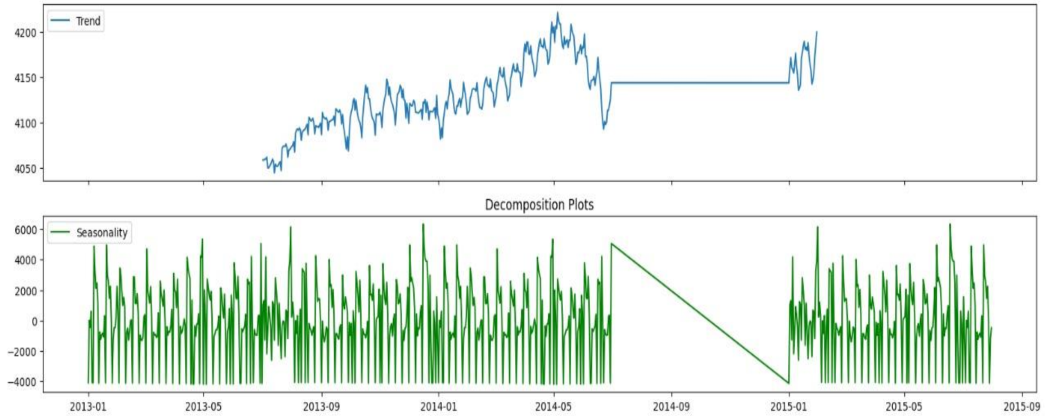
Store a



Store b



Store c



Store d

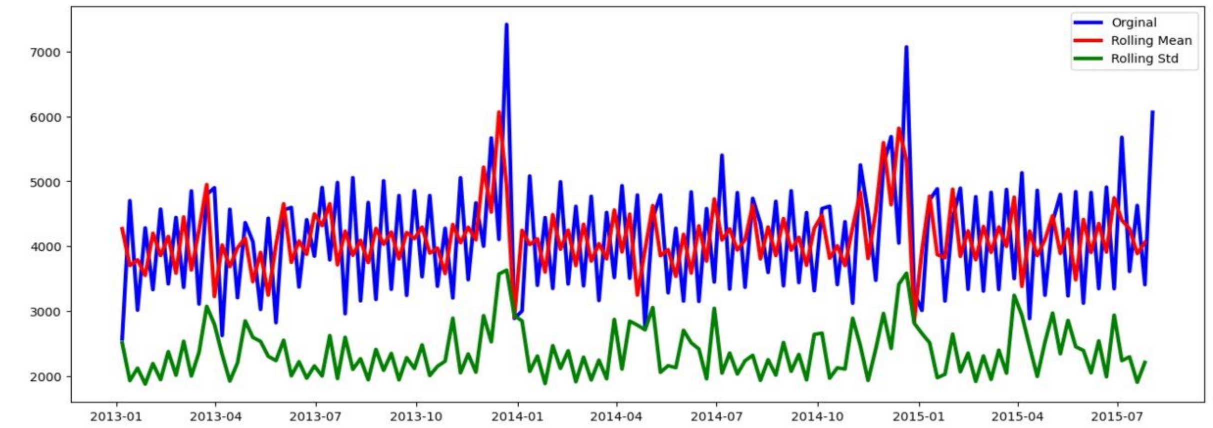
ADF Results & Stationarity Testing

- **ADF TEST RESULTS**

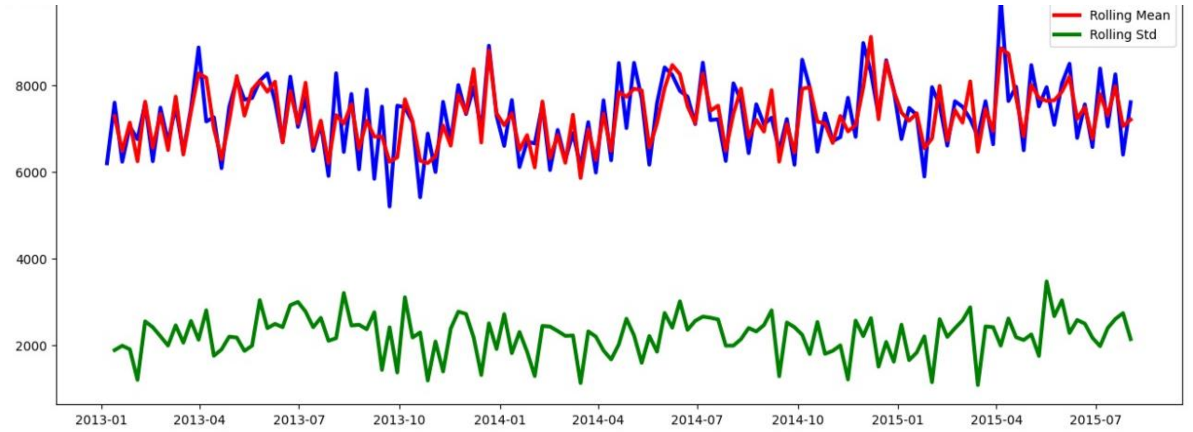
Store	ADF Stat	P-val	Crit 1%	Crit 5%	Crit 10%	Station?
a	-6.218237	0.000000	-3.437477	-2.864686	-2.568445	yes
b	-5.660918	0.000001	-3.437485	-2.864690	-2.568447	yes
c	-4.374784	0.000329	-3.437477	-2.864686	-2.568445	yes
d	-6.237461	0.000000	-3.439253	-2.865469	-2.568862	yes

ADF Results & Stationarity Testing

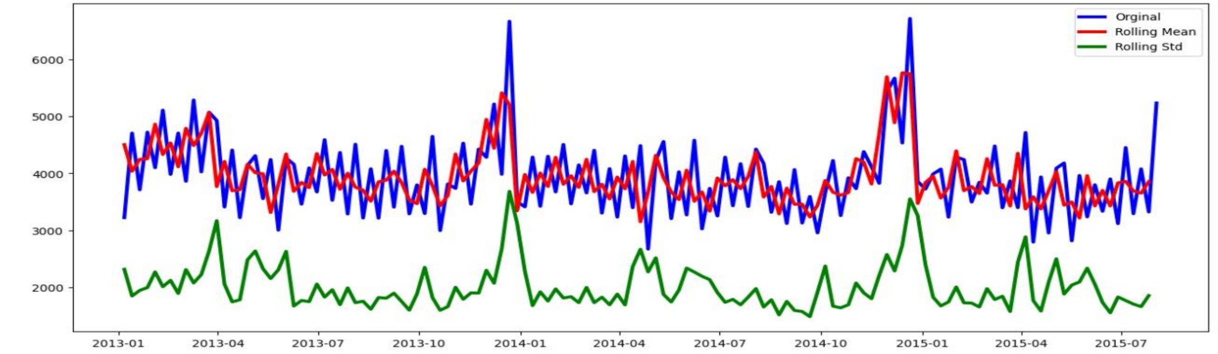
- **STATIONARITY PLOTS**



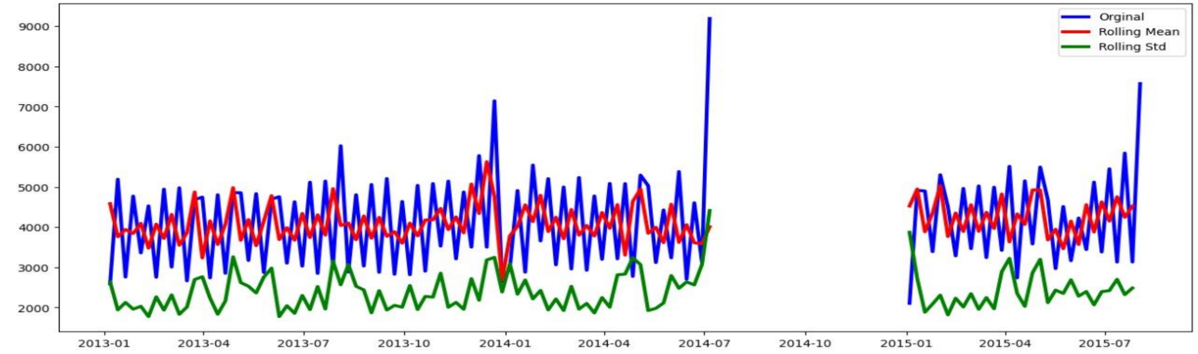
Store a



Store b

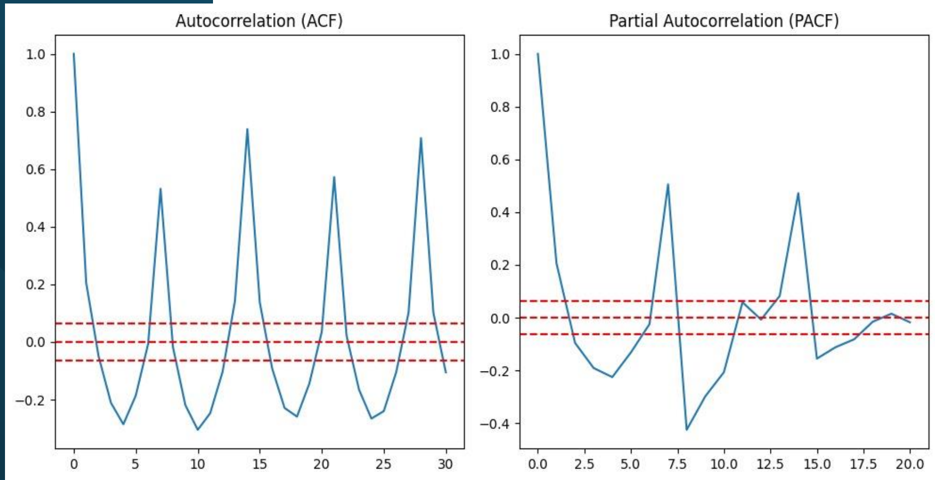


Store c

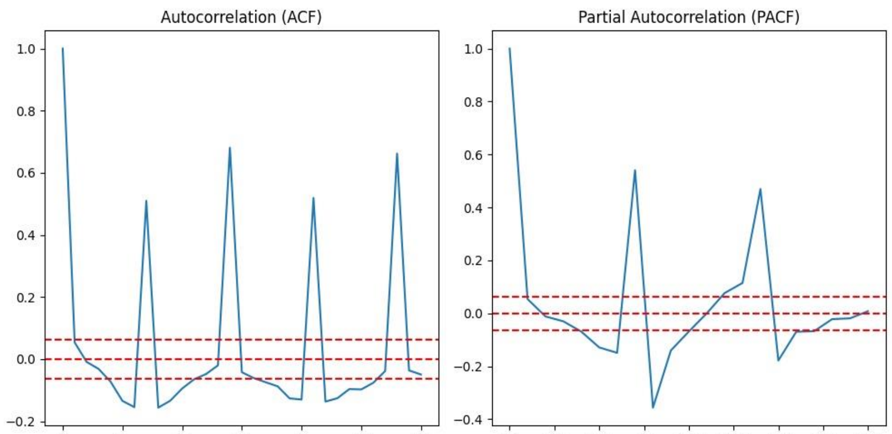


Store d

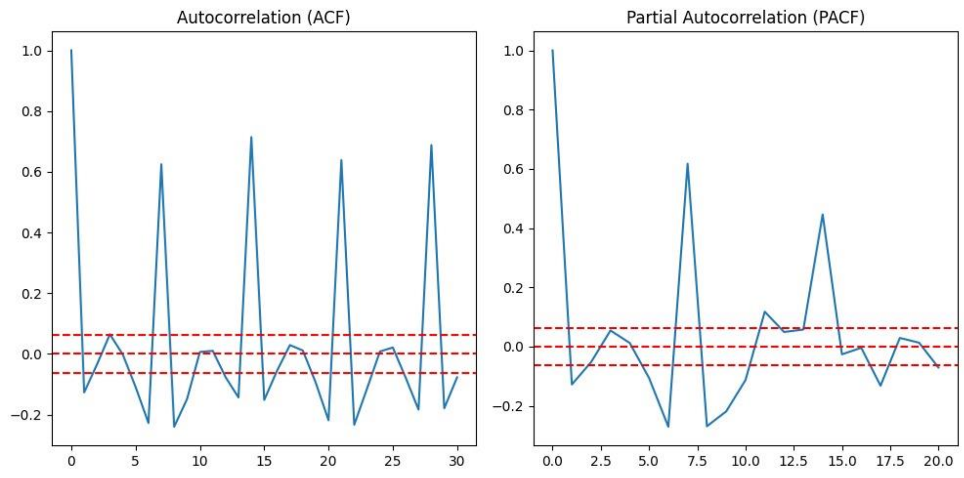
Autocorrelation & Partial Autocorrelation



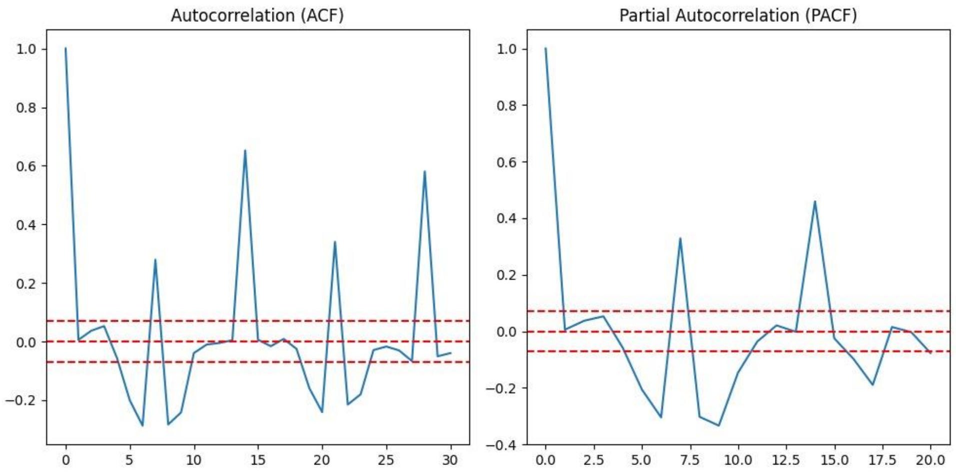
Store a



Store b



Store c



Store d

Modeling Approach

Model Approach Overview

- In our project, we tested three different models to forecast sales, and the goal was to compare their performance and select the most accurate one.

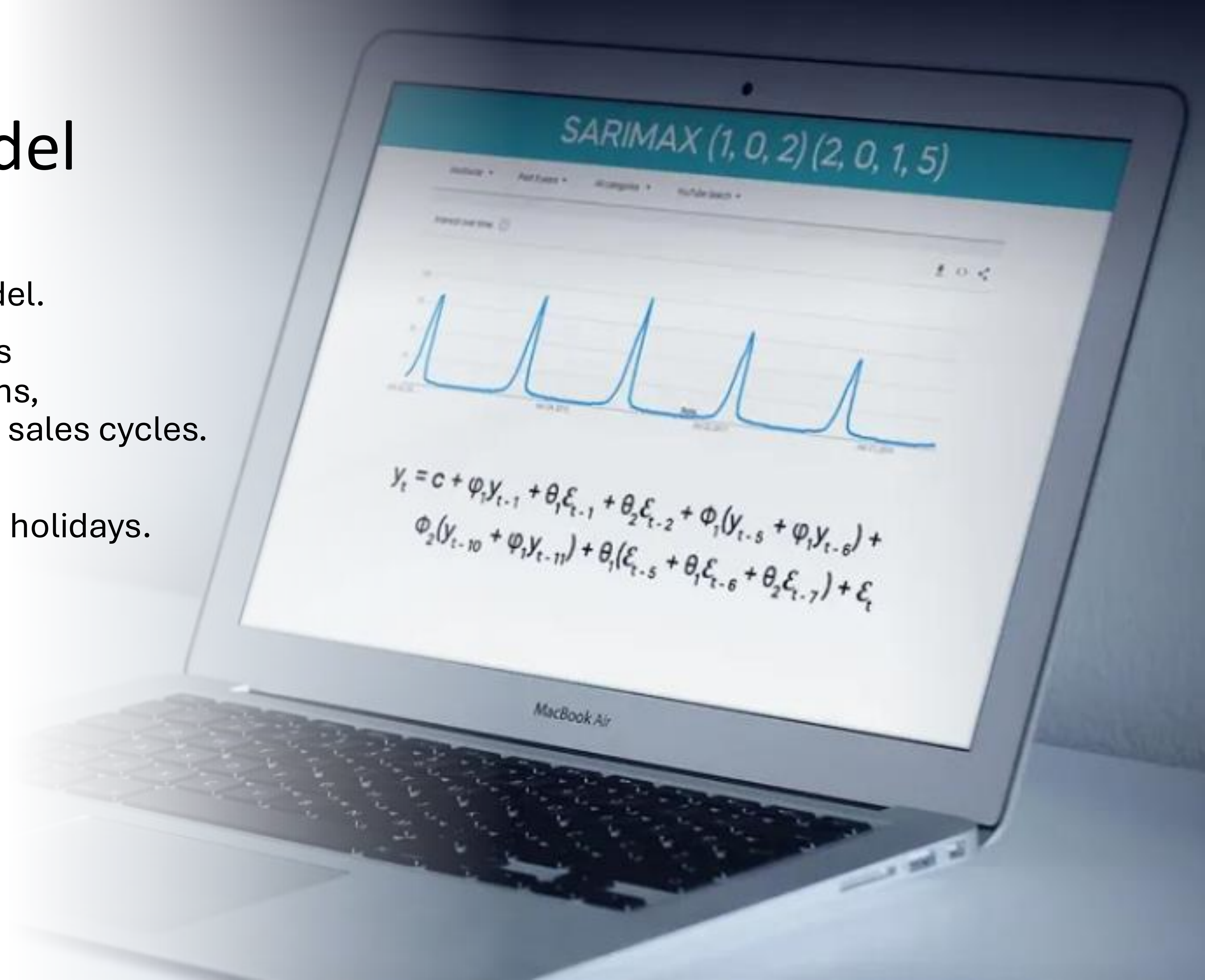
Why Multiple Models?

- We choose to test different types of models because each one captures different patterns in the data.

- Time-series models understand seasonality, while machine-learning models capture complex interactions.

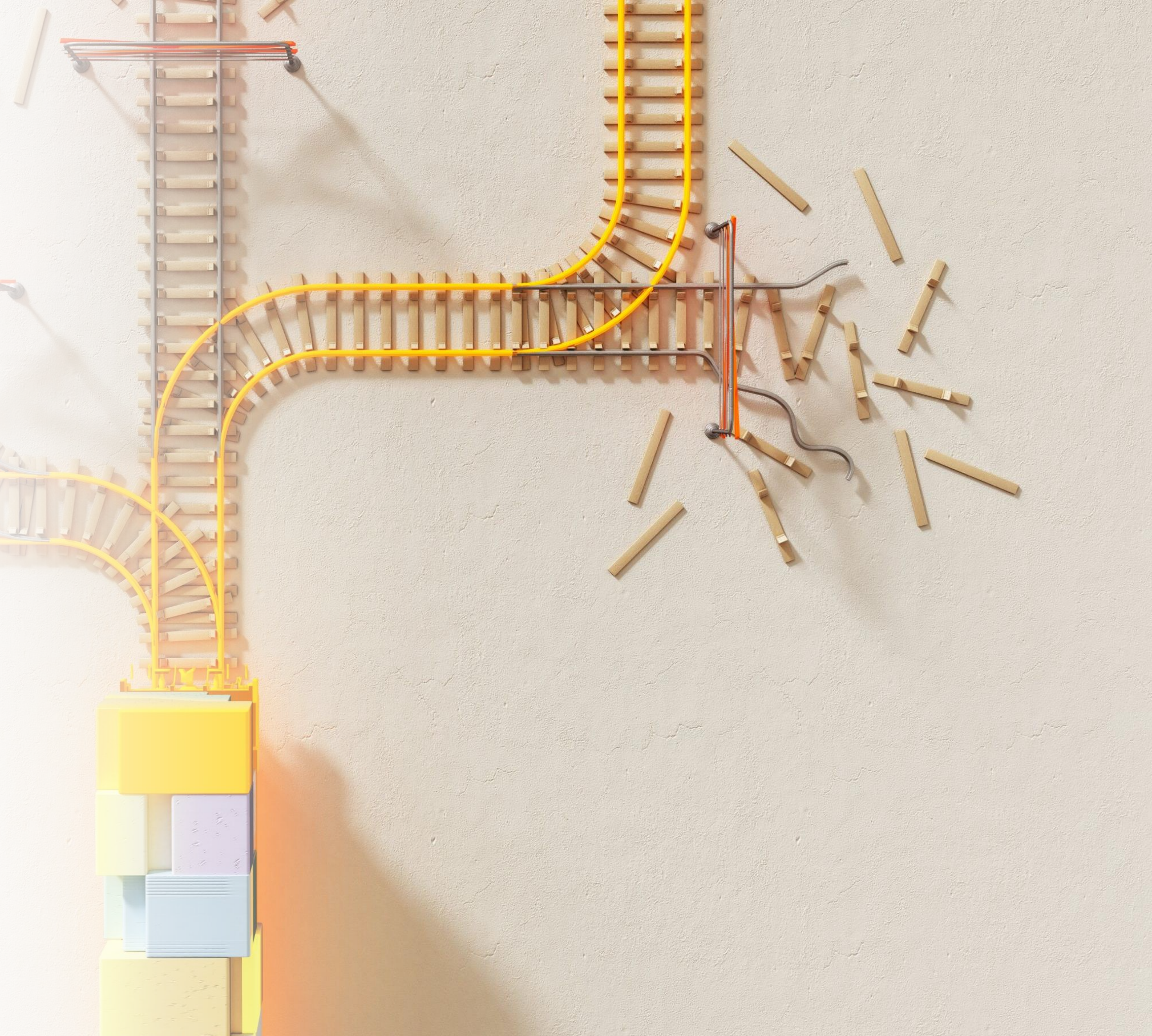
SARIMAX Model

- A classical time-series model.
- It captures long-term trends and strong seasonal patterns, such as yearly and monthly sales cycles.
- It allows us to add external factors like promotions and holidays.



XGBoost Model

- A machine-learning model that handles non-linear relationships.
- It works well when we provide engineered features like promotion duration, competition duration, and date features.
- It is slower to train compared to LightGBM.



LightGBM Model

- A faster and more optimized version of gradient boosting.
- It trains much faster than XGBoost and gave us better accuracy than XGBoost.
- LightGBM delivered the second-best performance overall.

Final Model Selection

- After comparing the results, SARIMAX achieved the lowest RMSE.
- So, we selected SARIMAX as our final model.

	SARIMA	XGBoost	LightGBoost
RMSE	867.95	1166.51	1056.42

Conclusion and Future Improvments

What is Project Deployment?

- Is the process of taking a developed model or application from a development environment and making it available for real-world use.

Benefits of Project Deployment

- **Real-Time Predictions**
  Allows models to generate predictions on live data instead of notebooks.

- **Business Impact**
  Transforms analytical insights into actionable decisions.

- **Scalability**
  Enables predictions for thousands of users or stores simultaneously.

- **Consistency & Reliability**
  Ensures the same model logic is used every time.

Conclusion

- The project achieved success in weekly sales forecasting using the Sarimax model.
- End user can use the webapp for forecasting and devs can use mlflow for experiment tracking.

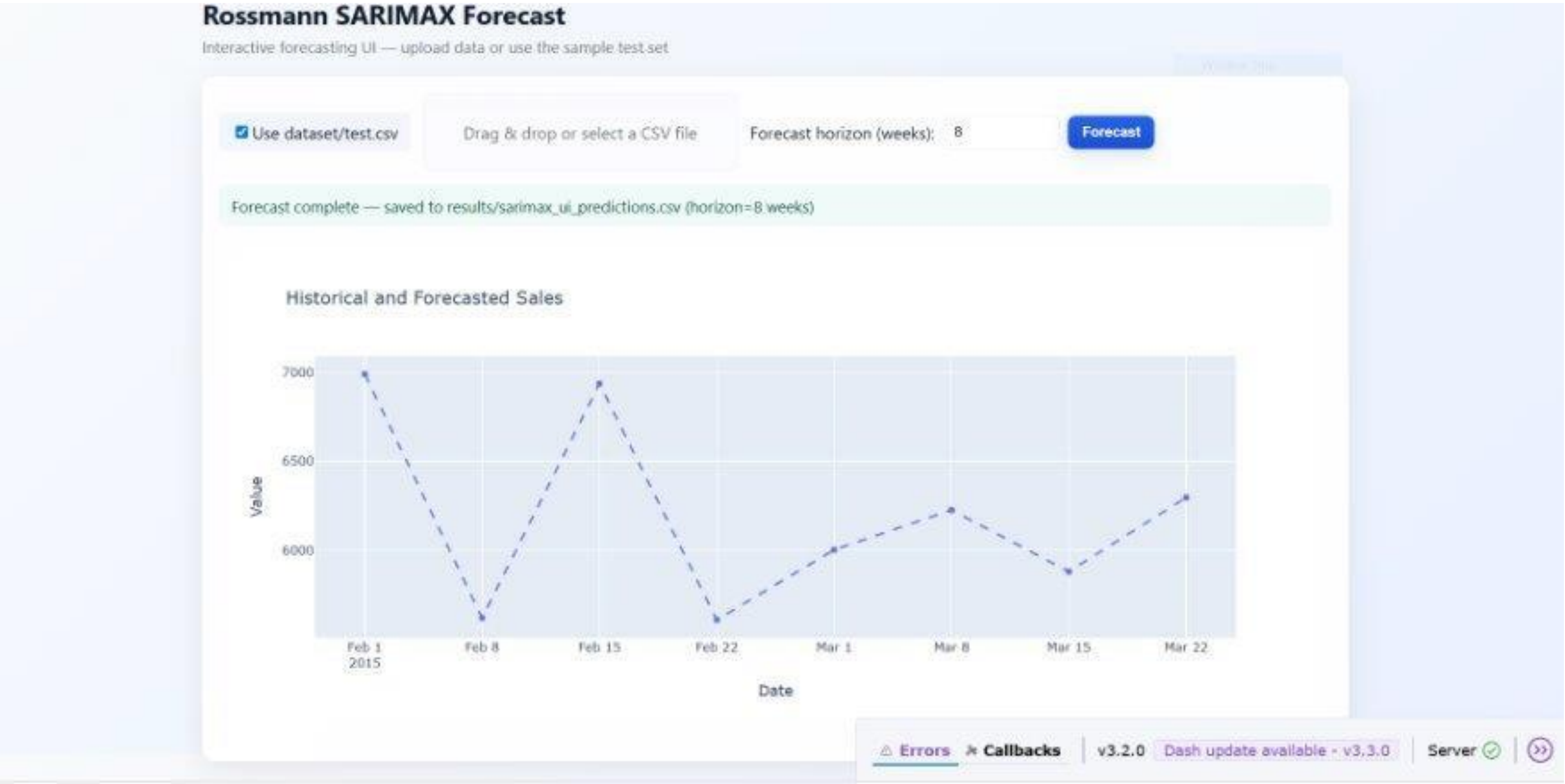
The project has been deployed from a notebook into multiple scripts and organized in the following way.

```
project-root/
│
├── dataset/                        # Training & validation datasets
│   ├── train.csv
│   ├── test.csv
│   └── store.csv
│
├── scripts/                        # Core ML & MLOps logic
│   ├── train.py                    # CLI training entrypoint (SARIMAX / LightGBM)
│   ├── predict.py                  # CLI inference helper (UI / batch jobs)
│   ├── data_loader.py              # Shared dataset loading & preprocessing logic
│   ├── lgbm_pyfunc.py               # MLflow pyfunc wrapper for LightGBM
│   ├── sarimax_pyfunc.py            # MLflow pyfunc wrapper for SARIMAX
│   └── list_mlflow_artifacts.py     # Inspect MLflow experiment artifacts
│
├── models/                         # Local serialized model artifacts
│   ├── lgb_artifacts.joblib
│   └── sarimax_artifacts.joblib
│
├── mlruns/                         # Local MLflow tracking store (dev only)
│
├── mlflow.db                      # Local MLflow metadata database
```

Project Structure

```
├── results/                          # Predictions & outputs used by UI
│   ├── sarimax_ui_predictions.csv
│   └── lgbm_predictions.csv
│
├── reports/                          # final project report
│   └── final_report.pdf
│
├── assets/                           # UI assets for Dash app
│   └── style.css
│
├── docs/                             # Project documentation
│   ├── deployment_structure.md       # Folder & deployment explanation
│   ├── EDA report.docx               # Explaratory data analysis
│   ├── Feature Engineering Summary.docx   # summary for all the newly added columns
│   ├── MLOps_Report.docx         # MLOps review report
│   └── monitoring_setup.md       # monitoring guide for the project
│
├── app.py                            # Dash application entrypoint
│
├── requirements.txt                  # Python dependencies
│
│
└── README.md                         # Project overview & quick start
```

Web App


Rossmann SARIMAX Forecast
Interactive forecasting UI — upload data or use the sample test set
Use dataset/test.csv
Drag & drop or select a CSV file
Forecast horizon (weeks):
8
Forecast
Forecast complete — saved to results/sarimax_ui_predictions.csv (horizon=8 weeks)
Historical and Forecasted Sales
7000
6500
6000
Value
Feb 1
2015
Feb 8
Feb 15
Feb 22
Mar 1
Mar 8
Mar 15
Mar 22
Date
Errors
Callbacks
v3.2.0
Dash update available - v3.3.0
Server

Room for Improvements

Improve RMSE and use other metrics

Deep learning models like LSTM, GRU, and Temporal Convolutional Networks

Room for Improvements

Per-Store Modeling

Incorporate lag features, rolling windows, weather data, macroeconomic indicators, and geospatial factors.

Deploy project on the cloud

What is MLflow

MLflow is an open-source platform used to manage the end-to-end machine learning lifecycle.

It helps data scientists track experiments, manage models, and deploy them consistently.

Why Do We Use MLflow?

- Machine learning experiments produce many models and parameters
- Hard to track :
- Which model performed best
- Which hyperparameters were used
- Which data version was trained on

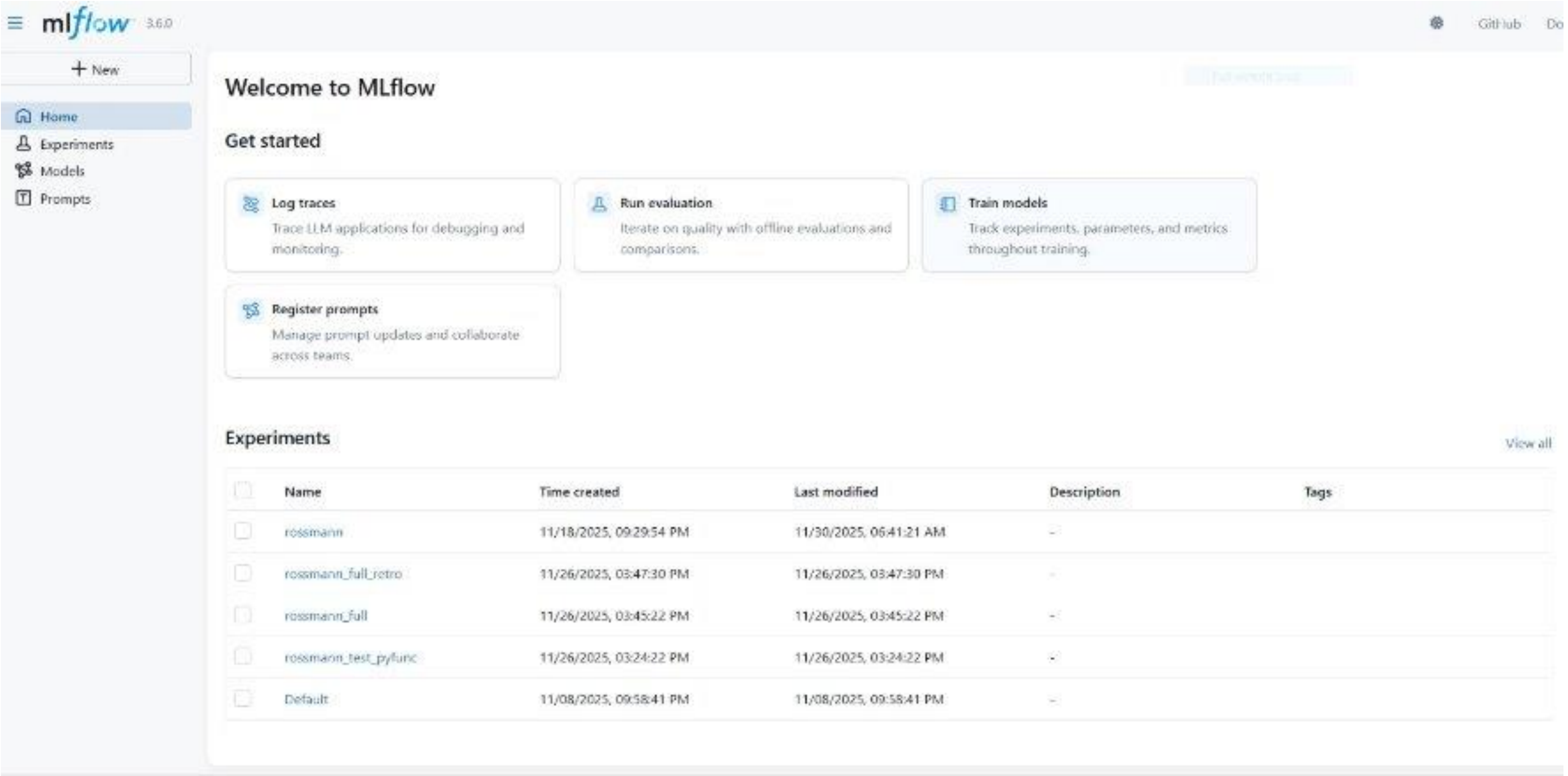
MLOps Flow Overview

- End-to-end pipeline for training, tracking, packaging, and serving models
- Artifacts saved locally then logged to MLflow
- Pyfunc wrappers ensure standardized inference
- Supports SARIMAX, and LightGBM models

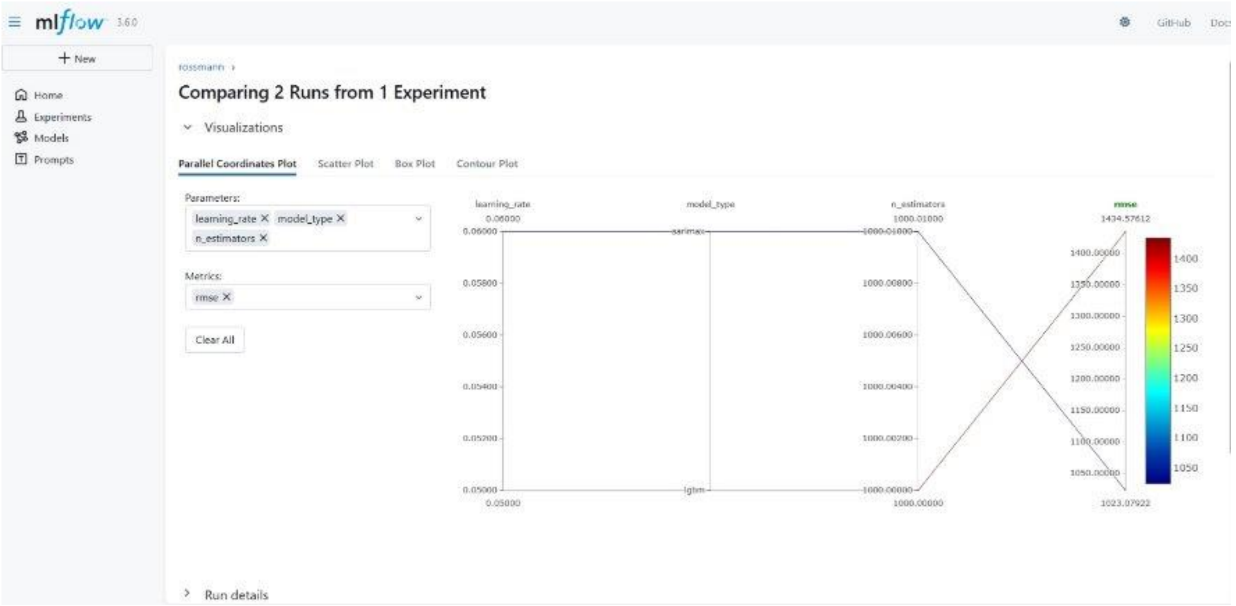
Experiment Tracking (MLflow UI)

- MLflow dashboard displaying experiments
- Organizes and audits model runs
- Tracks parameters, metrics, and artifacts
- Core tool for reproducible ML workflows

MLflow UI


mlflow
3.6.0
GitHub
New
Home
Experiments
Models
Prompts
Welcome to MLflow
Get started
Log traces
Trace LLM applications for debugging and monitoring.
Run evaluation
Iterate on quality with offline evaluations and comparisons.
Train models
Track experiments, parameters, and metrics throughout training.
Register prompts
Manage prompt updates and collaborate across teams.
Experiments
View all
Name
Time created
Last modified
Description
Tags
rossmann
11/18/2025, 09:29:54 PM
11/30/2025, 06:41:21 AM
rossmann_full_retro
11/26/2025, 03:47:30 PM
11/26/2025, 03:47:30 PM
rossmann_full
11/26/2025, 03:45:22 PM
11/26/2025, 03:45:22 PM
rossmann_test_pyfunc
11/26/2025, 03:24:22 PM
11/26/2025, 03:24:22 PM
Default
11/08/2025, 09:58:41 PM
11/08/2025, 09:58:41 PM

Model Comparison



THANK YOU

FOR YOUR APPRECIATION


ANY QUESTIONS ?